基于 NEURAL ODE 的复杂动力学学习

郑盼盼[*]
202321250043

龙裕坤[†]
202321250039

申晓达[‡]
202321250041

ABSTRACT

neuralODE 是一种将常微分方程和神经网络相结合的新方法。在神经网络中通过定义一个连续的动力系统来代替传统的离散层，这样可以在深度学习模型中使用连续时间模型。在本作业中，我们利用 `torchdiffeq` 这一利用 `torch` 来求解微分方程的第三方包。相继完成了对于简单时间序列的动力学学习、利用 neuralODE 实现连续的标准化流、使用 neuralODE 复现因果涌现，并在最后利用 neuralODE 对于心率数据进行分类，均得到了较好的结果。

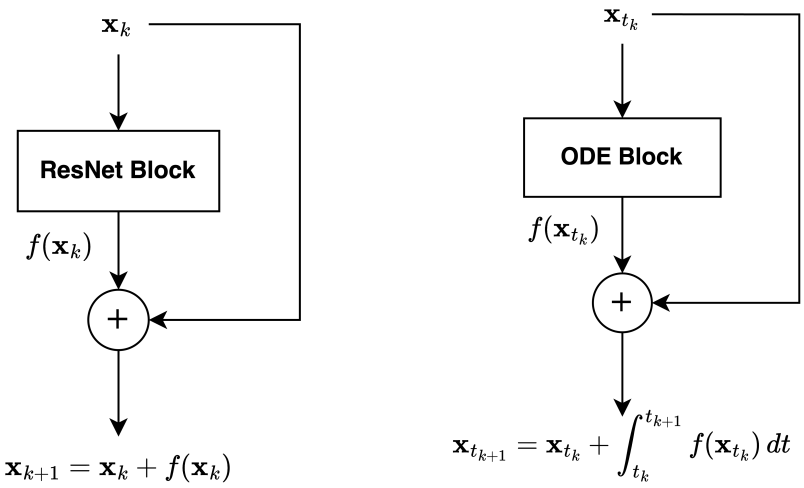
***K*eywords** neuralODE · normalizing flow · OT-flow · causality emergence

1 引言

neural ODE 为 Chen 等人于 2018 年所提出的一种结合常微分方程的深度学习概念。neural ODE 的思想源自于截距网络 (residual network, ResNet)，可被视为 ResNet 的一种推广：ResNet 由一层层的 `ResBlock` 堆叠而成。以第 k 层 `ResBlock` 为例（如图1a所示），其输入 x_k 后输出（相同维度的）$f(\mathrm{x}_k)$ 然后将 x_k 同 $f(\mathrm{x}_k)$ 相加得到下一层的输入刺激 $\mathrm{x}_{k+1} = \mathrm{x}_k + f(\mathrm{x}_k)$，因此，我们可以将此过程视为一个时间步为 1 的 Euler 法求微分方程数值解的过程；neural ODE 就是基于这一思想，将 ResNet 推广而来，其输入 x_{t_k} 输出其随时间演化的微分方程 $\dot{\mathrm{x}} = f(\mathrm{x}_{t_k})$ 然后使用相对更加精确的求解器，如 Runge-Kutta，来解出到下一时刻 t_{k+1} 的数值解 $\mathrm{x}_{t_{k+1}} = \mathrm{x}_{t_k} + \int_{t_k}^{t_{k+1}} f(\mathrm{x}_{t_k})\,dt$。因此，可以将 NerualODE 视为一种连续化的 ResNet。[1] 因此，可以将 neural ODE 视为一个黑箱，给一个输入 x，其就能给出这个输入随时间的演化规则 $\dot{\mathrm{x}}$，故可以用于学习比较复杂的演化过程。在此复杂动力学学习这部分，我们主要直接使用 neural ODE 来学习一个二维的动力系统。在此过程中 neural ODE 展现出了相对较好的表现。

除了对于复杂动力学的学习，neural ODE 还可以与标准化流 (normalizing flow) 相结合。标准化流一种生成模型，其大致思想就是利用一系列的双射 (bijective) $T : \mathbb{R}^d \to \mathbb{R}^d$ 将服从任意 d 维分布的随机变量 x 转化为服从 d 维正态分布的随机变量 u。且由于 T 为双射，我们可以使用逆函数 T^{-1} 来将服从 d 维正态分布的随机变量 u 转化成服从任意 d 维分布的随机变量 x，从而实现生成服从任意分布的数据的目的。我们可以使用 neural ODE 将原本离散的流转变成连续的流，我们通过 neural ODE 来学习随机变量 $\mathrm{z}(\mathrm{x}, t)$ 在连续的时间 $t \in [t_0, t_1]$ 上的演化规则 $\dot{\mathrm{z}}(\mathrm{x}, t)$，由于学习的是在时间上的演化规则，我们可以通过将时间 t_1 和 t_0 倒过来，从 $\mathrm{z}(\mathrm{x}, t_1)$ 求解出 $\mathrm{z}(\mathrm{x}, t_1)$ 的状态；而不需要像离散流那样需要构建特定的符合双射条件的神经网络（如 `RealNVP`）。在此部分，我们复现了连续化的 Planar 流 [2] 对于简单二维分布的学习和生成，同时也复现了结合运输问题 (Transport Optimization) 的标准化流：OT-Flow[3]。

[*]使用 neuralODE 实现连续标准化流、因果涌现论文的研讨
[†]使用 neuralODE 完成因果涌现的复现、利用 neuralODE 对心率信号进行分类
[‡]nerualODE 对于简单时间序列的动力学学习、因果涌现论文的研讨



(a) Illustration of ResNet

(b) Illustration of neural ODE

图 1: ResNet 和 neural ODE 的对比

信息压缩 (Information Squeeze) 的目标是从含有噪音的高维微观状态中恢复出未知的真实宏观状态, 张江老师在其工作中 [4] 将信息压缩（或称粗粒化）这一过程视为微观水平到宏观水平的一种因果涌现。在这一工作中，张江老师提出了一种名为：Neural Information Squeezer (NIS) 的机器学习框架实现上述过程。这一框架中包括一组可逆，结构相同且共享所有参数的编码器与解码器，以及一个基于普通前馈神经网络的宏观状态动力学学习器。事实上这一过程可以通过 neural ODE 实现。我们针对张江老师的工作进行了两个层次的复现：首先，基于普通前馈神经网络的宏观动力学显然可以替换为一个 neural ODE 结构，较之普通的前馈神经网络，neural ODE 显式地计算下一个时间步的宏观状态，具有更好的可理解性；在此基础之上，由于 neural ODE 具有可逆性，因此我们可以将编码器与解码器替换成一个 neural ODE 结构。我们的结果显示，上述两种使用 neural ODE 对结构进行了不同程度替换的 NIS 在信息压缩这一目标上与原文都有着近似的效果。

除此之外，neural ODE 还可以应用于分类问题的特征提取过程。我们应用了一个包含降采样模块，基于 neural ODE 特征提取模块以及全连接模块的深度学习结构对 MIT-BIH 心律失常数据库（MIT-BIH Arrhythmia Database）中的五类心律失常心电图 (ECG) 进行了分类，取得了平均 94.02% 的准确率。

2 方法[§]

2.1 neuralODE 对简单动力学的学习

神经常微分方程（Neural Ordinary Differential Equations，简称 Neural ODE）是一种基于常微分方程（Ordinary Differential Equations，ODEs）的深度学习方法，它结合了传统的 ODE 数值求解技术和神经网络模型。通过使用 ODE 来建模数据的演化过程，Neural ODE 可以自动地学习数据的动力学特征，从而可以进行预测、插值和生成等任务。Neural ODE 不再指定隐藏层的离散序列，而是使用神经网络参数化隐藏状态的导数。网络的输出是使用黑盒微分方程求解器计算的。这些连续深度模型具有恒定的内存成本，可以根据每个输入自适应其评估策略，并且可以明确地在速度和数值精度之间进行交换。neural ODE 模型开发用于解决多个目标问题: 1. 极深网络的优化问题 2. 用微分方程求解替代反向传播。

[§]具体代码见 `https://github.com/PanpanZheng-BNU/neuralODE`

2.1.1 ODE

常微分方程是一个只包含单个自变量 x 、未知函数 $f(x)$ 和其导数 $f'(x)$ 的等式。例如，$f'(x) = 2x$ 就是一个常微分方程的实例。更普遍的形式可以表示为 $\frac{df(x)}{dx} = g(f(x), x)$，其中 $g(f(x), x)$ 是由 $f(x)$ 和 x 构成的某种表达式。这种形式是扩展一般神经网络的关键。

对于常微分方程，通常的目标是求出未知的 $f(x)$。例如，$f'(x) = 2x$ 的通解是 $f(x) = x^2 + C$，其中 C 是任意常数。但在实际应用中，更常使用数值解法，这时给定初始值 $f(x_0)$，他们只需近似求出 $f(x_1)$，而不必得到完整的 $f(x)$ 解析式。

回到传统神经网络，其本质上是一个复杂的复合函数，复合的次数即网络的深度。例如，一个两层的全连接网络可以表示为 $Y = f(f(X, \theta_1), \theta_2)$。因此，每个神经网络层都可以看作是一个通用函数逼近器。

由于整体网络是由复合函数组成，所以使用链式法则进行求导是很直观的。梯度从最外层逐渐传递到内部层，并且每到一层都会更新该层的参数 θ。但是，前向传播后需要保存所有层的激活值，然后在反向传播时使用这些激活值计算梯度，这对内存的占用较大，从而限制了深度模型的训练。

Neural ODE 选择了不同的方法。它通过神经网络参数化隐藏状态的导数，而不是直接参数化隐藏状态本身。这里的导数参数化构建了连续的层级结构，而不是传统的离散层级。因此，参数空间也变得连续，不再需要分层传递梯度和更新参数。总的来说，Neural ODE 在前向传播中不存储任何中间结果，仅需常数级的内存成本。

2.1.2 从 RES-Net 到 Neural ODE

残差网络（RES-Net）是卷积网络的一种特殊形式，它的主要特点是采用了残差连接，解决了深度神经网络中的梯度消失问题。这使得即使在网络层级极深的情况下，梯度也能有效地传回至输入层。

残差网络中的核心模块是残差块，其结构包括直接的输入信息和内部卷积运算的输出信息的融合。这种结构保证了深层的模型在准确度上不会低于浅层的网络。当多个这样的残差模块被堆叠时，我们就得到了一个深度的残差神经网络。我们将图 1a 中的残差模块更加形式化地表示为以下方程：

$$h_{t+1} = h_t + f(h_t, \theta_t) \tag{1}$$

数学上，设 h_t 是第 t 层的输出，而 f 是由 θ_t 参数化的神经网络。整个残差模块可以用以下方程表示：$h_t{+}1{-}h_t = f(h_t, {}_t)$。这说明 f 参数化的是隐藏层之间的残差，并不直接参数化隐藏层。

进一步思考，如果将网络看作是连续的，而非离散的层，那么残差项 $(h_t + 1 - h_t)/1$ 将接近隐藏层对 t 的导数，其中的“1”表示两个离散层级之间的差异。这意味着神经网络实际上参数化了这个导数。

当我们试图在层级之间添加更多的层，并使之趋近于无限时，神经网络将呈现出连续性。从数学上说，残差网络可以看作是连续变换的欧拉离散化版本。这种连续变换可以使用常微分方程来表示：

$$\frac{dh(t)}{dt} = f(h(t), t, \theta) \tag{2}$$

换句话说，当 t 的变化接近于无限小时，隐藏状态的变化 $dh(t)$ 可以通过神经网络来建模。当 t 从初始状态渐变至终止状态时，$h(t)$ 的变化就代表了前向传播的结果。

要求得这个常微分方程的数值解，就相当于完成了前向传播。例如，如果输入图像为 $h(0) = X$，那么最终的隐藏层输出 $h(T)$ 就是推断结果。这是一个常微分方程的初值问题，可以使用常微分方程求解器（ODE Solver）来解决。而这种求解器可以控制数值误差，允许我们在计算效率和模型准确度之间进行权衡。

形式上来说，现在就需要变换方程 (2) 以求出数值解，即给定初始状态 $h(t_0)$ 和神经网络的情况下求出终止状态 $h(t_1)$：

$$\frac{dh(t)}{dt} = f(h(t), t, \theta) \tag{3}$$

$$\int_{t_0}^{t_1} dh(t) = \int_{t_0}^{t_1} f(h(t), t, \theta)dt \tag{4}$$

总的来说，我们只需要给定初始状态 $h(t_0)$ 和神经网络的情况，就可以求出终止状态 $h_(t_1)$。这需要对神经网络 f 从 t_0 到 t_1 的部分进行积分。幸运的是，我们可以直接使用在数学物理领域已经很成熟的 ODE solver 来得到这个值。

前面提到过残差网络是神经常微分方程的特例，可以说残差网络是欧拉方法的离散化。两三百年前解常微分方程的欧拉法非常直观，即 $h(t+t) = h(t)+t\times f(h(t), t)$。每当隐藏层沿 t 走一小步 δt，新的隐藏层状态 $h(t+t)$ 就应该近似在已有的方向上迈一小步。如果这样一小步一小步从 $t0$ 走到 $t1$，那么就求出了 ODE 的数值解。如果我们令 t 每次都等于 1，那么离散化的欧拉方法就等于残差模块的表达式 $h(t+1) = h(t) + f(h(t), t)$。但是欧拉法只是解常微分方程最基础的方法，它每走一步都会产生一点误差，且误差会累积起来。近百年来，数学家构建了很多现代 ODE 求解方法，它们不仅能保证收敛到真实解，同时还能控制误差水平。

陈天琦等研究者构建的 ODE 网络 [1] 就使用了一种适应性的 ODE solver，它不像欧拉法移动固定的步长，相反它会根据给定的误差容忍度选择适当的步长逼近真实解。如下图所示，左边的残差网络定义有限转换的离散序列，它从 0 到 1 再到 5 是离散的层级数，且在每一层通过激活函数做一次非线性转换。此外，黑色的评估位置可以视为神经元，它会对输入做一次转换以修正传递的值。而右侧的 ODE 网络定义了一个向量场，隐藏状态会有一个连续的转换，黑色的评估点也会根据误差容忍度自动调整。

在与 ResNet 的类比中，我们基本上已经了解了 ODEnet 的前向传播过程。首先输入数据 $Z(t_0)$，我们可以通过一个连续的转换函数（神经网络）对输入进行非线性变换，从而得到 f。随后 ODESolver 对 f 进行积分，再加上初值就可以得到最后的推断结果。如下所示，残差网络只不过是用一个离散的残差连接代替 ODE Solver。

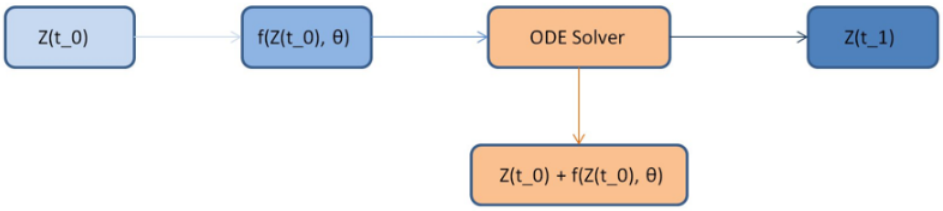


图 2: 在前向传播中，ODEnet 还有几个非常重要的性质，即模型的层级数与模型的误差控制。首先因为是连续模型，其并没有明确的层级数，因此我们只能使用相似的度量确定模型的深度，作者在这篇论文中采用 ODE Solver 评估的次数作为深度。

正如作者而言，训练一个连续层级网络的主要技术难点在于令梯度穿过 ODE Solver 的反向传播。其实如果令梯度沿着前向传播的计算路径反传回去是非常直观的，但是内存占用会比较大而且数值误差也不能控制。作者的解决方案是将前向传播的 ODE Solver 视为一个黑箱操作，梯度很难或根本不需要传递进去，只需要绕过就行了。作者采用了一种名为 adjoint method 的梯度计算方法 [1] 来绕过前向传播中的 ODE Solver，即模型在反传中通过第二个增广 ODE Solver 算出梯度，其可以逼近按计算路径从 ODE Solver 传递回的梯度，因此可用于进一步的参数更新。这种方法不仅在计算和内存非常有优势，同时还能精确地控制数值误差。

2.1.3 动力学学习

接下来，我们通过复现官方文档的案例：使用 torchdiffeq 学习一个动力学模型来模拟一个螺旋 ODE：

$$\dot{y}_t = Ay_t^3 \tag{5}$$

方程 5 定义了 forward 方法，它指定了 ODE 的具体形式。对于每个时间点 t 和状态 y，forward 方法返回 y 的三次方与系数矩阵 A 的矩阵乘积，其中

$$A = \begin{bmatrix} -0.1 & -2.0 \\ 2.0 & -0.1 \end{bmatrix}$$

此外，ODE 求解器采取的是 5 阶 Runge-Kutta 方法，并且采用了伴随的方法以节省内存。具体的 function 定义为 2 个输入 50 个输出的线性层，结合 tanh 激活函数，其它超参数见代码注释。

2.2 标准化流

标准化流 (noarmalizing flow) 是一种生成模型，其目标是是寻找一个双射 (bijection) $T : \mathbb{R}^d \to \mathbb{R}^d$，将初始 d 维随机变量 u（一般这一随机变量服从 d 维标准正态分布），转变为一个服从新分布 $p(\mathrm{x})$ 的新随机变量 x，用数学形式表示为

$$\begin{aligned} \mathrm{x} &= T(\mathrm{u}) \\ \mathrm{u} &= T^{-1}(\mathrm{x}) \end{aligned} \tag{6}$$

而这一双射往往由一系列的小双射 T_k 组成，即：

$$T = T_N \circ T_{N-1} \circ \cdots \circ T_1$$

因此，其被称为**流**，而**标准化**指的是我们将任意的 d 维分布标准化为 d 维的正态分布。当我们对随机变量进行变化时，其概率密度 $p(\mathrm{x})$ 会进行相应的变化：

$$\log p(\mathrm{x}) = \log p(\mathrm{u}) - \log|\det J_T(\mathrm{u})| \tag{7}$$

其中 $J_f(\mathrm{u})$ 是函数 $f \in \mathbb{R}^d$ 对于变量 $\mathrm{u} \in \mathbb{R}^d$ 的雅可比矩阵 (Jacobian Matrix)

$$J_T(\mathrm{u}) \coloneqq \begin{bmatrix} \frac{\partial T}{\partial u_1} & \cdots & \frac{\partial T}{\partial u_d} \end{bmatrix} = \begin{bmatrix} \frac{\partial T_1}{\partial u_1} & \cdots & \frac{\partial T_1}{\partial u_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial T_d}{\partial u_1} & \cdots & \frac{\partial T_d}{\partial u_d} \end{bmatrix}$$

对于离散流而言，这一双射 T 需要特定的神经网络结构，如 `RealNVP`，来实现其可逆的性质。

2.2.1 连续流

在连续流中我们一般使用一个向量 $\mathrm{z}_t \in \mathbb{R}^d$ 来描述在时刻 t 时流的状态。一般令初始状态 z_{t_0} 为服从基础分布的随机变量 u 而最终状态 z_{t_1} 为服从目标分布的随机变量 x。我们的目标是学习将初始随机变量转化为目标随机变量的演化规则 $g_{\boldsymbol{\phi}}(t, \mathrm{z}_t) = \dot{\mathrm{z}_t}$。当然，参考（离散）标准化流的基本定义：6和7，我们不仅需要一个微分方程来表述随机变量的演化过程，还需要一个微分方程来描述随机变量概率密度的演化过程。

1 **随机变量的演化**：我们使用 z_t 表示流在时刻 t 的状态。假定时间 t 在 t_0 到 t_1 上连续，使得 $\mathrm{z}_{t_0} = \mathrm{u}$ 而 $\mathrm{z}_{t_1} = \mathrm{x}$。我们可以构建出一个参数为 $\boldsymbol{\phi}$ 的方程 $g_{\boldsymbol{\phi}} : \mathbb{R}^d \to \mathbb{R}^d$（此处的 g 由神经网络构成），即

$$\frac{d\mathrm{z}_t}{dt} = g_{\boldsymbol{\phi}}(t, \mathrm{z}_t)$$

为了得到演化后的结果 x 我们需要对此 ODE 进行求解

$$\mathrm{x} = \mathrm{z}_{t_1} = \mathrm{u} + \int_{t_0}^{t_1} g_{\boldsymbol{\phi}}(t, \mathrm{z}_t)\, dt = \mathrm{ODESolve}(\mathrm{u}, g_{\boldsymbol{\phi}}, t_0, t_1)$$

若我们可以直接得到服从目标分布 $p(\mathrm{x})$ 的随机变量 x，我们可以对以上过程反方向求解，得到服从基础分布的随机变量 u：

$$\mathrm{u} = \mathrm{z}_{t_0} = \mathrm{x} + \int_{t_1}^{t_0} g_{\boldsymbol{\phi}}(t, \mathrm{z}_t)\, dt = \mathrm{x} - \int_{t_0}^{t_1} g_{\boldsymbol{\phi}}(t, \mathrm{z}_t)\, dt = \mathrm{ODESolve}(\mathrm{x}, g_{\boldsymbol{\phi}}, t_1, t_0)$$

因此，不同于离散流中要求变换 T 必须是可逆映射，**在连续流中，对于神经网络 g 并没有可逆的苛刻要求**。

2 **随机变量概率密度的对数的演化**：Chen 等人证明，在连续流中，随机变量的概率密度随时间的演化为其雅各比矩阵的迹的相反数 [1]，即

$$\frac{d \log p(\mathrm{z}_t)}{dt} = -\mathrm{tr}\{J_{g_{\boldsymbol{\phi}}(t,\cdot)}(\mathrm{z}_t)\}$$

其中，$J_{g_{\boldsymbol{\phi}}(t,\cdot)}(\mathrm{z}_t)$ 为 $g(\mathrm{z}_t)$ 对 z_t 中每个元素求偏导的雅各比矩阵。通过对于动力系统求解，我们有

$$\log p(\mathrm{x}) = \log p(\mathrm{u}) - \int_{t_0}^{t_1} \mathrm{tr}\{J_{g_{\boldsymbol{\phi}}(t,\cdot)}(\mathrm{z}_t)\}\, dt$$

综上，我们可以将连续流的演化过程用如下的方程组进行描述：

$$\begin{bmatrix} \mathrm{x} \\ \log p(\mathrm{x}) \end{bmatrix} = \begin{bmatrix} \mathrm{u} \\ \log p(\mathrm{u}) \end{bmatrix} + \int_{t_0}^{t_1} \begin{bmatrix} g_{\boldsymbol{\phi}}(t, \mathrm{z}_t) \\ -\mathrm{tr}\{J_{g_{\boldsymbol{\phi}}(t,\cdot)}(\mathrm{z}_t)\} \end{bmatrix} dt \tag{8}$$

2.2.2 连续 Planar 流

Rezende 等人于 2015 年提出了基于 ResNet 流的一种转化函数 [2]（将任意目标分布 $p_1(\mathrm{x})$ 转化为 d 维标准正态分布 $p_0(\mathrm{x})$），

$$\mathrm{u}, \mathrm{w} \in \mathbb{R}^d, b \in \mathbb{R}$$
$$f^{-1}(\mathrm{x}) = \mathrm{x} + \mathrm{u} \cdot \tanh(\mathrm{w}^T \mathrm{x} + b)$$

其中 $\mathrm{w}, \mathrm{u}, b$ 由一个神经网络 HyperNet : $\mathbb{R} \to \mathbb{R}^{2d+1}$ 所得到：即给该神经网络输入一个时间 t 其能返回 $\mathrm{w}, \mathrm{u}, b$ 这三个值。然后我们再将利用所得的这三个值，构建一个微分方程组：

$$\begin{bmatrix} \frac{d\mathrm{z}_t}{dt} \\ \frac{\partial \log p(\mathrm{z}_t)}{\partial t} \end{bmatrix} = \begin{bmatrix} \mathrm{u} \cdot \tanh(\mathrm{w}^T \mathrm{z}_t + b) \\ -\mathrm{tr}\{J(\mathrm{z}_t)\} \end{bmatrix} \tag{9}$$

其中 $J(\mathrm{z}_t)$ 为 $\mathrm{u} \cdot \tanh(\mathrm{w}^T \mathrm{z}_t + b)$ 对 z_t 的雅各比矩阵。在实际实现过程中，直接通过调用 `torch.autograd` 实现。假设我们可以从目标随机分布 $p(\mathrm{x})$ 中进行采样，并将得到的随机变量定义为最终状态 z_{t_1}，并令 $\log p(\mathrm{z}_{t_1}) = 0$，结合公式9所定义的微分方程组，在给定 t_0, t_1 的条件下，我们可以对这一微分方程组进行求解，得到初始状态 z_{t_0}，其对应服从基础分布（标准正态分布）$p(\mathrm{u})$ 的随机变量 u，即

$$\begin{bmatrix} \mathrm{z}_{t_0} \\ \log p(\mathrm{z}_{t_0}) \end{bmatrix} = \begin{bmatrix} \mathrm{z}_{t_1} \\ \log p(\mathrm{z}_{t_1}) \end{bmatrix} + \int_{t_1}^{t_0} \begin{bmatrix} \mathrm{u} \cdot \tanh(\mathrm{w}^T \mathrm{z}_t + b) \\ -\mathrm{tr}\left\{J(\mathrm{z}_t)\right\} \end{bmatrix} dt$$

根据此微分方程组的解，我们使用如下的损失函数来描述 z_{t_0} 服从正态分布的程度（等价于 KL 散度)

$$l = \mathbb{E}_{p(\mathrm{u})} \left\{ \frac{1}{2} \|\mathrm{z}_{t_0}\|^2 + \log p(\mathrm{z}_{t_0}) + \frac{d}{2} \log(2\pi) \right\}$$

具体算法如A.1所示。

2.2.3 OT Flow

Onken 等人将标准化流同运输问题 (Optimal Transport, OT) 相结合，提出了 OT 流的概念。其主要在损失函数上对于连续流做了一定的优化，相较于最简单的用于描述从目标分布随机变量转化出的随机变量服从基础分布的程度。其引入了两个正则项，对于每个随机变量的运动距离做出了约束（即当随机变量的运动路程过长时，损失函数也会相对较大）。[3]

我们分别用 ρ_0 和 ρ_1 描述任意目标分布和标准正态分布（基础分布）。使用 $\mathrm{z}(\mathrm{x},t)$ 描述在 t 时刻 t 的随机变量的状态。令 $\mathrm{z}(\mathrm{x},0)$ 为服从目标分布的随机变量 x，而 $\mathrm{z}(\mathrm{x},T)$ 为服从标准正态分布的随机变量 u。作者提出了如下的微分方程组

$$\partial_t \begin{bmatrix} \mathrm{z}(\mathrm{x},t) \\ \ell(\mathrm{x},t) \\ L(\mathrm{x},t) \\ R(\mathrm{x},t) \end{bmatrix} = \begin{bmatrix} -\nabla\Phi(\mathrm{z}(\mathrm{x},t),t;\boldsymbol{\theta}) \\ -\mathrm{tr}(\nabla^2\Phi(\mathrm{z}(\mathrm{x},t),t;\boldsymbol{\theta})) \\ \frac{1}{2}\|\nabla\Phi(\mathrm{z}(\mathrm{x},t),t;\boldsymbol{\theta})\|^2 \\ \left|\partial_t\Phi(\mathrm{z}(\mathrm{x},t),t;\boldsymbol{\theta}) - \frac{1}{2}\|\nabla\Phi(\mathrm{z}(\mathrm{x},t),t;\boldsymbol{\theta})\|^2\right| \end{bmatrix}, \begin{bmatrix} \mathrm{z}(\mathrm{x},0) \\ \ell(\mathrm{x},0) \\ L(\mathrm{x},0) \\ R(\mathrm{x},0) \end{bmatrix} = \begin{bmatrix} \mathrm{x} \\ 0 \\ 0 \\ 0 \end{bmatrix} \tag{10}$$

其中，x 为服从目标分布 $\rho_0(\mathrm{x})$ 的随机变量。基于如上的方程组，有如下的损失函数 $\mathcal{L}(\mathrm{x},T)$

$$\begin{aligned} \mathcal{L}(\mathrm{x},T) &= \mathbb{E}_{\rho_0(\mathrm{x})}\left\{\alpha_1 C(\mathrm{x},T) + L(\mathrm{x},T) + \alpha_2 R(\mathrm{x},T)\right\} \\ C(\mathrm{x},T) &\coloneqq \frac{1}{2}\|\mathrm{z}(\mathrm{x},T)\|^2 - \ell(\mathrm{x},T) + \frac{d}{2}\log(2\pi) \end{aligned} \tag{11}$$

该损失函数中：

- $C(\mathrm{x},T)$：描述随机变量 $\mathrm{z}(\mathrm{x},T)$ 服从标准正态分布的程度。
- $L(\mathrm{x},T)$：用于描述随机变量的运输成本 (Transport Cost)。
- $R(\mathrm{x},T)$：由 HJB Equation 所得到。

其中 $\Phi(\mathrm{s};\boldsymbol{\theta})$ 为作者所构建的一个势函数 (Potential function)：

$$\begin{aligned} \Phi(\mathrm{s};\boldsymbol{\theta}) = \mathrm{w}^T N(\mathrm{s};\boldsymbol{\theta}) + \frac{1}{2}\mathrm{s}^T(A^TA)\mathrm{s} + \mathrm{b}^T\mathrm{s} + c, \\ \text{where } \boldsymbol{\theta} = (\mathrm{w},\boldsymbol{\theta}_N,A,\mathrm{b},c). \end{aligned} \tag{12}$$

其中，

- $\mathrm{s} = [\mathrm{x},t] \in \mathbb{R}^{d+1}$，即将随机变量 x 同时间 t 拼接所得的向量。
- $N : \mathbb{R}^{d+1} \to \mathbb{R}^m$ 为一个 ResNet，其输入一个 $d+1$ 维的向量并返回一个 m 维的向量。
- 参数 $\boldsymbol{\theta}$ 包含所有可训练的神经网络权重：$\mathrm{w} \in \mathbb{R}^m, \boldsymbol{\theta}_N \in \mathbb{R}^p, A \in \mathbb{R}^{r\times(d+1)}, \mathrm{b} \in \mathbb{R}^{d+1}, c \in \mathbb{R}$。令 $r = \min(10,d)$ 来限制实对称矩阵 A^TA 的参数数目。

ResNet 作者在论文中提到其实际使用了一个两层的 ResNet，而且更高的 ResNet 层数并没有明显的提升，第一层负责将 $d+1$ 维的向量映射到 m 维上，第二层负责在 m 维的隐空间内进行变换：

$$\begin{aligned} \mathrm{u}_0 &= \sigma(K_0\mathrm{s} + \mathrm{b}_0) \\ N(\mathrm{s};\boldsymbol{\theta}_N) = \mathrm{u}_1 &= \mathrm{u}_0 + h\sigma(K_1\mathrm{u}_0 + \mathrm{b}_1) \end{aligned} \tag{13}$$

其中，步长 (step-size) $h = 1$，密度矩阵 $K_0 \in \mathbb{R}^{m\times(d+1)}$，$K_1 \in \mathbb{R}^{m\times m}$，偏置 $\mathrm{b}_0, \mathrm{b}_1 \in \mathbb{R}^m$。我们选择对于逐元素 (element-wise) 的激活函数 $\sigma(\mathrm{x}) = \log(\exp(\mathrm{x}) + \exp(-\mathrm{x}))$，其是双曲正切函数的不定积分，即 $\sigma'(\mathrm{x}) = \tanh(\mathrm{x})$。因此，双曲正切函数是流 $\nabla\Phi$ 的激活函数。

梯度计算 势函数 Φ 的梯度为

$$\nabla_{\mathrm{s}}\Phi(\mathrm{s};\boldsymbol{\theta}) = \nabla_{\mathrm{s}}N(\mathrm{s};\boldsymbol{\theta}_N)\mathrm{w} + (A^TA)\mathrm{s} + \mathrm{b} \tag{14}$$

其中，我们使用前 d 维作为 d 维随机变量 $\mathrm{z}(\mathrm{x},t)$ 随时间的变化量（对应公式10中的 $\nabla\Phi(\mathrm{z}(\mathrm{x},t;\boldsymbol{\theta}))$。而梯度中的最后一项对应于势函数对于时间 t 的偏导：$\partial_t\Phi(\mathrm{z}(\mathrm{x},t;\boldsymbol{\theta}))$ 其被用于描述 $R(\mathrm{x},t)$ 随时间的变化量。对于第一项 $\nabla_{\mathrm{s}}N(\mathrm{s};\boldsymbol{\theta}_N)\mathrm{w}$，我们可以通过链式法则计算：

$$\begin{aligned}
&\mathrm{z}_1 = \mathrm{w} + hK_1^T\mathrm{diag}(\sigma'(K_1\mathrm{u}_0 + \mathrm{b}_1))\mathrm{w},\\
&\mathrm{z}_0 = K_0^T\mathrm{diag}(\sigma'(K_0\mathrm{s} + \mathrm{b}_0))\mathrm{z}_1, \quad \text{where}\\
&\nabla_{\mathrm{s}}N(\mathrm{s};\boldsymbol{\theta}_n)\mathrm{w} \leftarrow \mathrm{z}_0
\end{aligned} \tag{15}$$

其中，diag 为将向量对角化，相当于对于 $\mathrm{diag}(\mathrm{q})\mathrm{x}$ 相当于对于两个相同维度的向量 q,x 进行逐元素 (element-wise) 相乘。

迹的计算 我们可以通过如下的方法来计算势函数海塞矩阵 (Hessian Matrix) 的迹：

$$\mathrm{tr}(\nabla^2\Phi(\mathrm{s};\boldsymbol{\theta})) = \mathrm{tr}\left(E^T\nabla_{\mathrm{s}}^2(N(\mathrm{s};\boldsymbol{\theta}_N)\mathrm{w})E\right) + \mathrm{tr}\left(E^T(A^TA)E\right) \tag{16}$$

其中，$E \in \mathbb{R}^{(d+1)\times d}$，其由 $d+1$ 维空间中，前 d 的标准基向量组成，即

$$E = \begin{bmatrix} 1 & 0 & \cdots & 0 & 0\\ 0 & 1 & \cdots & 0 & 0\\ \vdots & \vdots & \ddots & \vdots & \vdots\\ 0 & 0 & \cdots & 1 & 0\\ 0 & 0 & \cdots & 0 & 1\\ 0 & 0 & \cdots & 0 & 0 \end{bmatrix} \in \mathbb{R}^{(d+1)\times d}$$

其主要通过 E^TQE 的方式选择出 $(d+1)\times(d+1)$ 维矩阵 Q 的前 d 行和前 d 列。我们可以使用如下方法来计算式16中的第一个部分 $\mathrm{tr}(E^T\nabla_{\mathrm{s}}^2(N(\mathrm{s};\boldsymbol{\theta}_N)\mathrm{w})E)$：

$$\begin{aligned}
&\mathrm{tr}(E^T\nabla_{\mathrm{s}}^2(N(\mathrm{s};\boldsymbol{\theta}_N)\mathrm{w})E) = t_0 + ht_1, \quad \text{where}\\
&t_0 = (\sigma''(K_0\mathrm{s} + \mathrm{b}_0)\odot\mathrm{z}_1)^T((K_0E)\odot(K_0E))1\\
&t_1 = (\sigma''(K_1\mathrm{u}_0 + \mathrm{b}_1)\odot\mathrm{w})^T((K_1J)\odot(K_1J))1
\end{aligned} \tag{17}$$

其中，$\odot$ 表示相同维度的向量或矩阵之间逐元素相乘 (element-wise multiply)，$1 \in \mathbb{R}^d$ 为一个元素全部为 1 的向量，用于将 d 维向量中的所有元素相加，$J = \nabla\mathrm{u}_0^T = \mathrm{diag}(\sigma'(K_0\mathrm{s} + \mathrm{b}_0))(K_0E)$。

综上，我们通过如上的方法来对于微分方程组 10 进行计算。详细算法如 A.2 所示。

2.3 信息压缩

信息压缩的这个问题来自系统科学学院张江老师探索深度学习能否涌现出因果关系，在这里不具体介绍因果涌现的定量表述，从定性的角度来说，当一个系统的宏观状态 (Macro State) 被隐藏进微观状态 (Micro State) 中，如果通过信息压缩的过程将隐藏的宏观状态恢复出来，我们就可以说发生了涌现的现象。

以简谐运动为例，考虑一个简单的弹簧振子系统，弹簧振子可以简单地由位置 z 及速度 v 定义，我们有动力学方程：

$$\begin{cases} \frac{dz}{dt} = v\\ \frac{dv}{dt} = -z \end{cases} \tag{18}$$

因此一个弹簧振子系统的状态可以被表述为 $\mathrm{x}=(z,v)$，这也就是弹簧振子的宏观状态现在假设利用两个传感器对这个弹簧振子系统进行观测，由于存在观测误差，因此观测得到的状态存在噪音，不妨设：

$$\begin{cases}\mathrm{x}_1=\mathrm{x}+\xi\\ \mathrm{x}_2=\mathrm{x}-\xi\end{cases} \tag{19}$$

其中 $\xi\sim\mathcal{N}(0,\mathrm{conv})$, 也即噪音服从二维高斯分布:

$$\mathrm{conv}=\begin{bmatrix}\delta_1 & 0\\ 0 & \delta_2\end{bmatrix} \tag{20}$$

我们将 x_1 与 x_2 联合起来，得到一个四维的向量，这就是定义的微观状态，信息压缩的目标就是从这个微观状态中恢复出真实但未知的宏观状态 x

为了实现这一目标，张江老师在其论文中设计了一种机器学习架构 NIS，端到端地训练这一机器学习架构能够自然地将未知的宏观状态通过编码的过程恢复出来，NIS 的主要结构包括：

- **编码器**：要求有可逆的结构，也即将其正向过程的输出作为逆向过程的输入，经过逆向过程的计算可以得到其正向过程的输入
- **宏观动力学学习器**：学习宏观状态的动力学形式，计算下一时间步的宏观状态
- **解码器**：与编码器有着完全相同的结构与参数，解码过程为编码过程的逆向过程

NIS 的模型结构如图3所示。

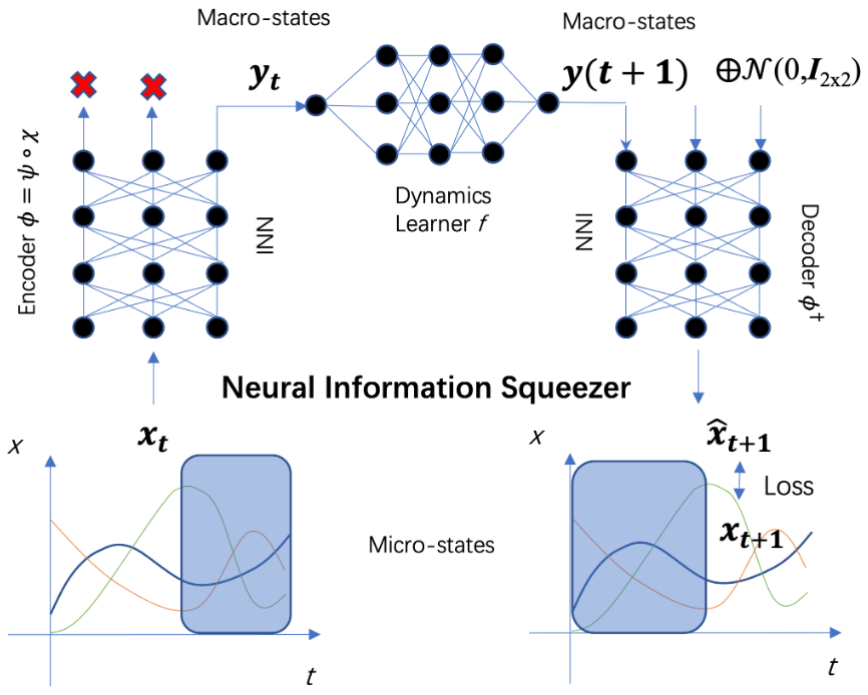


图 3: NIS 的主要结构 [4]

该模型首先将 t 时间步的微观状态 x_t 作为一个编码器的输入，完成编码的过程，编码器的输出为 ϕ，人为舍弃编码器输出的 s 个维度（例如弹簧振子这一任务中输入向量为四维，因此可以选择舍弃一维，两维或者三

维），得到预测的宏观状态 y_t，在理想情况下预测的宏观状态 y_t 与真实宏观状态相同或呈现偏置为 0 的线性关系。为了通过端到端的训练达到这一效果，该框架利用一个宏观动力学学习器 f 学习到 y_t 的动力学形式，依此得到下一个时间步的宏观状态 y_{t+1}（原文中由一个简单的前馈神经网络实现）。为 y_{t+1} 补充 s 维度的高斯噪声 $\mathcal{N}$，作为解码器的输入。解码器与编码器结构相同，但计算方向是逆向的。解码器解码出预测的下一时间步的微观状态 $\hat{x}_{t+1}$，计算预测的微观状态与实际微观状态 x_{t+1} 之间的损失函数（这里选择了均方误差 MSE），并通过反向传播过程对模型进行训练，理想的情况下随着损失函数的下降，这一过程能够使得编码器从微观状态中编码出宏观状态。

2.3.1 neural ODE 作为宏观动力学学习器

由于张江老师的工作中宏观动力学学习器为普通的前馈神经网络，这一部分自然地可以替换为 neural ODE 结构，基于 neural ODE 的宏观动力学学习器主要结构如图4。

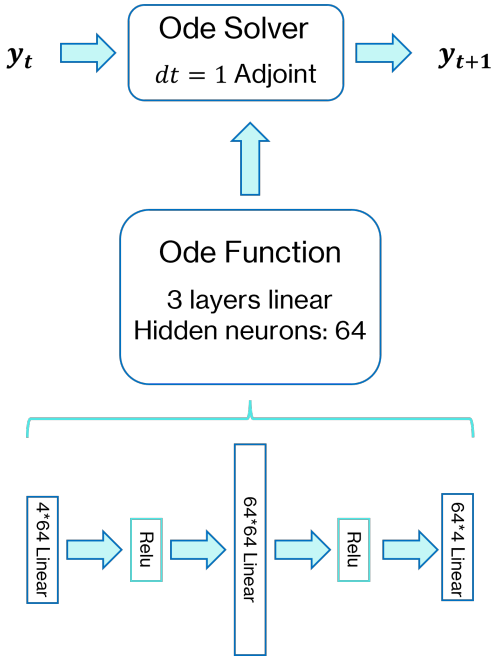


图 4: neural ODE 宏观动力学学习器

基于 neural ODE 的宏观动力学学习器具有两个主要部分：

- **ODE Function**：一个三层的前馈神经网络，具有 64 个隐藏神经元，用于描述宏观状态之间的动力学形式
- **ODE Solver**：一个求解器，用于求解下一时间步的宏观状态，采用了 adjoint 方法节省内存开销

由于在第一层次的复现中我们只将宏观动力学学习器替换为了 neural ODE 的结构，因此保留了张江老师原文中的编码器与解码器。在原工作中，这一部分采用的编码器与解码器结构为 RealNVP，最早由 Dinh 等人在 2017 年提出 [5]。原文中用到的 RealNVP 结构如图5。

RealNVP 是一个完全可逆的网络结构，其中 s,t 均为三层的前馈神经网络，且 s_1,s_2 共用参数，t_1,t_2 同样共用参数。RealNVP 首先将输入向量 x 分成两个部分 x_1, x_2（在这里用掩膜实现），在第一阶段，向量 x_2 保持不变，并将其作为前馈神经网络 s_1 与 t_1 的输入，向量 x_1 与前馈神经网络 s_1 的输出做逐元素的乘法，与 t_1 的输出做逐元素加法得到向量 x_1'，为了保证模型的可逆性，这里的乘法与加法事实上是指数计算：

$$x_1' = x_1 * e^{s_1} + t_1 \tag{21}$$

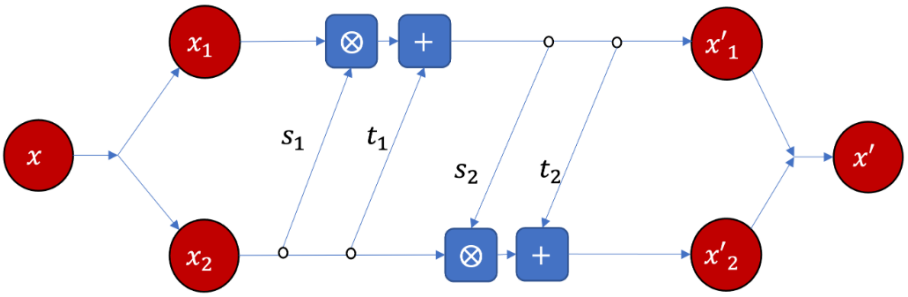


图 5: RealNVP 的主要结构 [4]

由于当前馈神经网络 s_1 的输出 s_1 为三位数时，e^{s_1} 会超过 pytorch 的浮点数计算上限，因此我们在这里设定前馈神经网络 s_1（以及 s_2 的激活函数) 为 tanh. 在第二阶段互换针对向量 x'_1 与向量 x_2 的操作，重复上述过程，得到向量 x'_2，最后合并 x'_1 与 x'_2 得到 x' 完成正向过程，我们将正向过程作为编码器。

不难发现，RealNVP 是可逆的，这也意味着 RealNVP 的逆向计算过程可以自然地作为我们模型的解码器。在逆向过程中 RealNVP 将输入的向量 x' 分为 x'_1 与 x'_2 两个部分，首先向量 x'_1 保持不变，并将其作为前馈神经网络 s_2 与 t_2 的输入，向量 x_2' 与前馈神经网络 s_2 的输出做逐元素的除法，与 t_1 的输出做逐元素减法得到向量 x_2，这里体现了使用指数运算的必要性，如果直接采用除法，当向量 s_2 中存在 0 值时会造成错误，指数计算如下：

$$\mathrm{x}_2 = (\mathrm{x}'_2 - \mathrm{t}_2) * e^{-\mathrm{s}_2} \tag{22}$$

逆向过程的第二阶段不再赘述。

至此，我们完成了使用 neural ODE 作为宏观动力学学习器，保留原工作中 RealNVP 作为编码器与解码器的 NIS 结构的介绍，这一模型在信息压缩任务中的效果将在本文中实验与结果的部分展现。

2.3.2 neural ODE 作为编码器与解码器

在用 neural ODE 替换原宏观动力学学习器的基础上，我们还对 NIS 的结构进行了进一步的变更：用 neural ODE 结构作为 NIS 中的编码器与解码器。

上文提到，NIS 架构要求编码器与解码器可逆，事实上 neural ODE 由于其性质正是完全可逆的。如上文所述，neural ODE 的正向过程可以视为：

$$\hat{\mathrm{x}} = \int_t^{t+1} f dt + \mathrm{x} \tag{23}$$

其中 f 为 Ode Function , 也即用神经网络反映的动力学形式，那么显然地，我们可以将 neural ODE 的逆向过程表述为：

$$\mathrm{x} = \int_{t+1}^{t} f dt + \hat{\mathrm{x}} \tag{24}$$

因此，我们可以将一个 neural ODE 结构作为可逆的编码器与解码器对原 NIS 中的 RealNVP 编码器/解码器进行替换，在这里使用的 neural ODE 编码器/解码器结构如图6。Neural ODE 编码器/解码器同样分为两个主要部分：

- **Ode Function**：一个六层的前馈神经网络，具有 64 个隐藏神经元，大体上我们将两个普通的三层前馈神经网络串接在一起组成 Ode Function 的主要结构
- **Ode Solver**：求解器是 neural ODE 编码器/解码器可逆的关键，其中编码过程为：$\hat{\mathrm{x}} = \int_t^{t+1} f dt + \mathrm{x}$，解码过程为：$\mathrm{x} = \int_{t+1}^{t} f dt + \hat{\mathrm{x}}$，需要注意的是，这里 Ode Solver 对下一个时间步的求解并不反映真实的下一时间步的状态，而是一种可逆的编码/解码方式，类似于特征提取的过程。同样采取 adjoint 方法节省内存开销

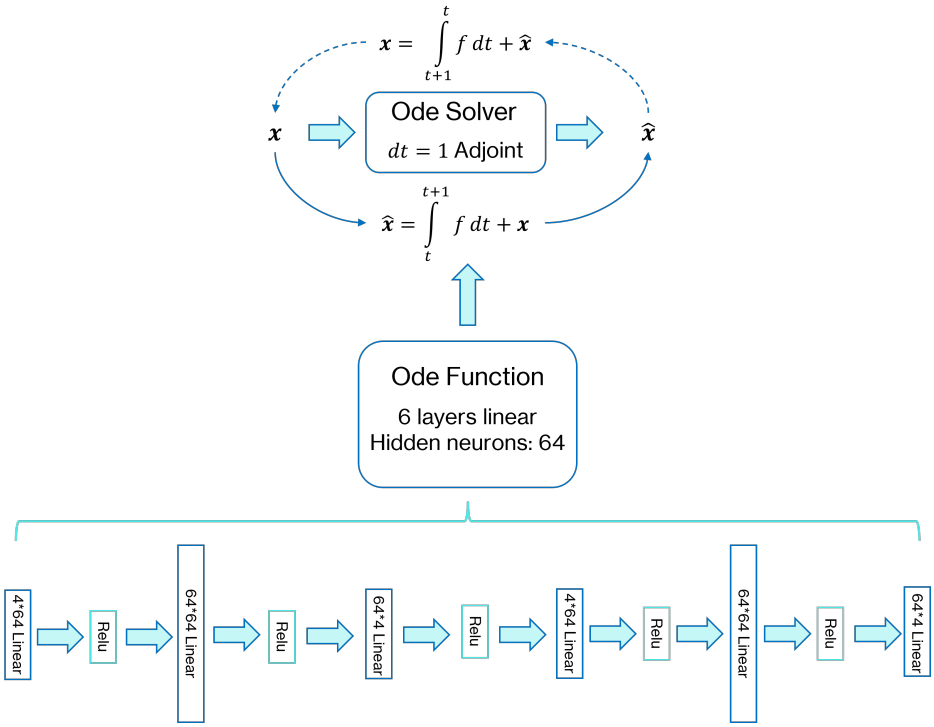


图 6: neural ODE 编码器/解码器结构

将 neural ODE 结构的编码器/解码器与 neural ODE 结构的宏观动力学学习器组合起来，我们就得到了完全基于 neural ODE 的 NIS 机器学习架构，这一架构在信息压缩任务上的性能同样将在实验与结果的部分展示。

2.4 neural ODE 用于分类问题

Neural ODE 不仅能对动力系统进行学习，还能够作为特征提取器为分类模型提供依据特征。

为了将 neural ODE 应用于分类过程，我们搭建了一个包含三个主要模块的分类模型如图7：

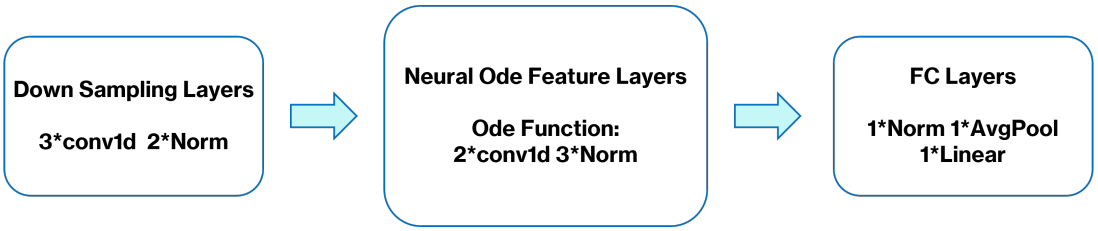


图 7: 利用 neural ODE 进行特征提取的分类模型

其中三个模块分别为：

- **Down Sampling Layers**：用于降采样，由于我们实际上是对时间序列数据进行分类，因此降采样过程主要由 3 个一维卷积层实现，在 3 个一维卷积层之间设置了标准化层以提高模型的学习效率，激活函数为 Relu
- **Neural Ode Feature Layers**：利用 neural ODE 对降采样的数据进行特征提取，其中
  1. Ode Function 包含 2 个一维卷积层，与 3 个标准化层交替串接，激活函数为 Relu

2. Ode Solver 求解该模块输入数据在 $dt = 1$ 个时间步之后的状态，这里的求解不反映输入数据在下一时间步的真实状态，而是作为特征提取的过程，采用了 adjoint 方法节省内存开销

- **FC Layers**：全连接层，包含 1 个标准化层，1 个平均池化层，以及 1 个线性层用于将展平的数据映射到维度为类别数量的输出向量上，向量中的每一个元素代表该输入数据属于对应类别的概率，在输出时利用 softmax 方法得到最终的预测类别

我们测试了这一分类模型在 MIT-BIH 心律失常数据库（MIT-BIH Arrhythmia Database）上的分类性能，具体结果将在实验与结果的部分展示。

3 实验与结果

3.1 复杂动力学学习

我们通过隔一段时间记录并绘制 ODE 学习的情况，发现从 iters = 20 到 iters = end，从基本没有学习到动力学到基本吻合，损失函数从 0.65 降到了 0.60

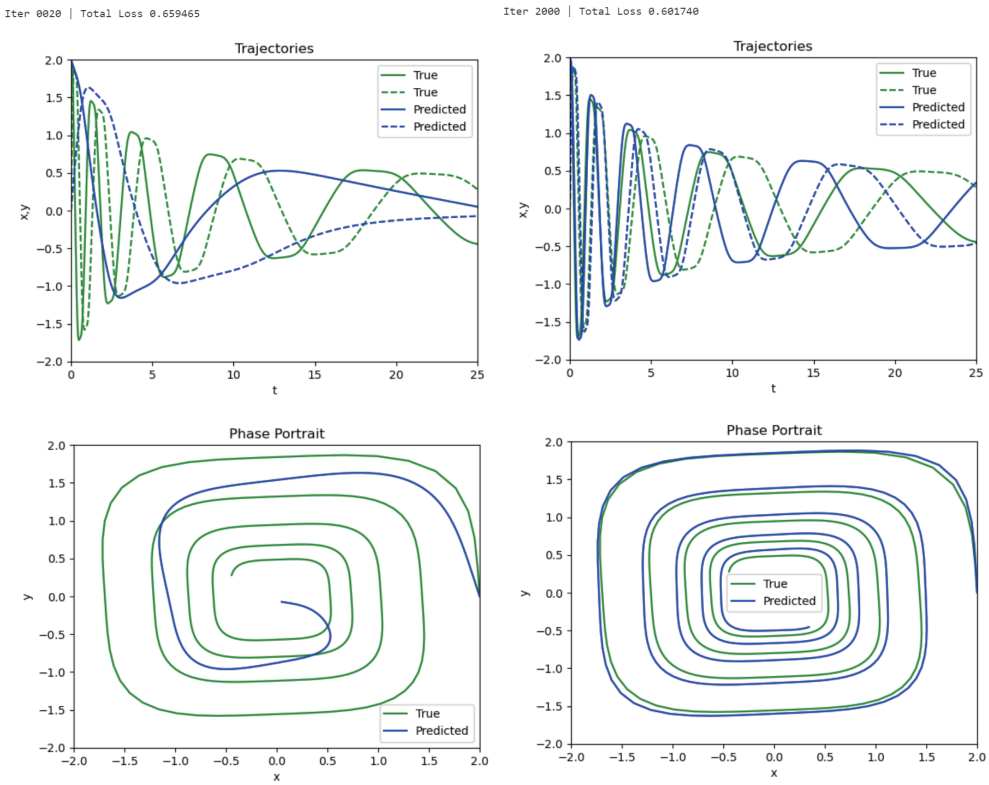


图 8: 展现了 neural ODE 学习动力系统的演化过程，从第 20 个轮次到最终结果，上下展示的是不同的观察视角，可以看到随着轮次的增加，neural ODE 学习到了真正的动力系统。

3.2 标准化流

我们分别用 Planar Flow 和 OT Flow 分别训练了三类 ToyDatasets (gaussians, moons and spheres) 的生成模型，大体训练和生成的流程如下：

1. 我们生成服从目标分布 (gaussians, moons 或 spheres) 的数据点 $\mathrm{x} \in \mathbb{R}^d$，然后将数据点作为初始时刻 t_0 的状态 $\mathrm{z}(\mathrm{x}, t_0) \in \mathbb{R}^d$，并根据流的类型生成增广向量 $\mathrm{aug_vec}(\mathrm{z}, t_0)$。具体而言，根据流的类型，增广矩阵的维度不同：对于 Planar Flow，其增广向量为 $d+1$ 维的向量，故 t_0 时刻增广向量为 $\mathrm{aug_vec}(\mathrm{z}, t_0) = [\mathrm{z}(\mathrm{x}, t_0); 0]$；其前 d 个维度表示随机变量；最后一维度表示随机变量的对数概率密度。而 OT Flow 的增广向量为 $d+3$ 维向量，t_0 时刻的增广向量为 $\mathrm{aug_vec}(\mathrm{z}, t_0) = [\mathrm{z}(\mathrm{x}, t_0); 0; 0; 0]$ 和 Planar Flow 的增广矩阵类似，其前 d 维度表示 d 维随机变量 z 而后三个维度对应于 eq.10 中的 ℓ, L, R。

2. 根据方法中介绍的流的定义构建增广矩阵 aug_vec 演化的方程组 ODEFunc

3. 利用 `torchdiffeq` 中的求解器，求的 t_1 时刻的增广矩阵 $\mathrm{aug_vec}(\mathrm{z}, t_1)$ ($\mathrm{aug_vec}(\mathrm{z}, t_1) = \mathrm{ODESolve}(\mathrm{aug_vec}(\mathrm{z}, t_1), \mathrm{ODEFunc}, t_0, t_1)$)，根据增广矩阵，计算 t_1 时刻的损失函数。并反向传播，优化 ODEFunc 中的参数。

4. 根据不断训练得到的 ODEFunc 我们将上面的求解过程反过来，即：先生成服从 d 维标准正态分布的数据点，并构建 t_1 时刻的增广向量 $\mathrm{aug_vec}(\mathrm{z}, t_1) = [\mathrm{z}(\mathrm{x}, t_1); 0...]$ （由于我们只需要生成数据，而不再训练模型，增广矩阵中除了前 d 维是我们的随机变量外，剩余维度均定为 0），然后我们对于增广矩阵进行反向的求解 $\mathrm{aug_vec}(\mathrm{z}, t_0) = \mathrm{ODESolve}(\mathrm{aug_vec}(\mathrm{z}, t_1), t_1, t_0)$ 即可生成服从目标分布的数据。

生成结果如下图9所示：

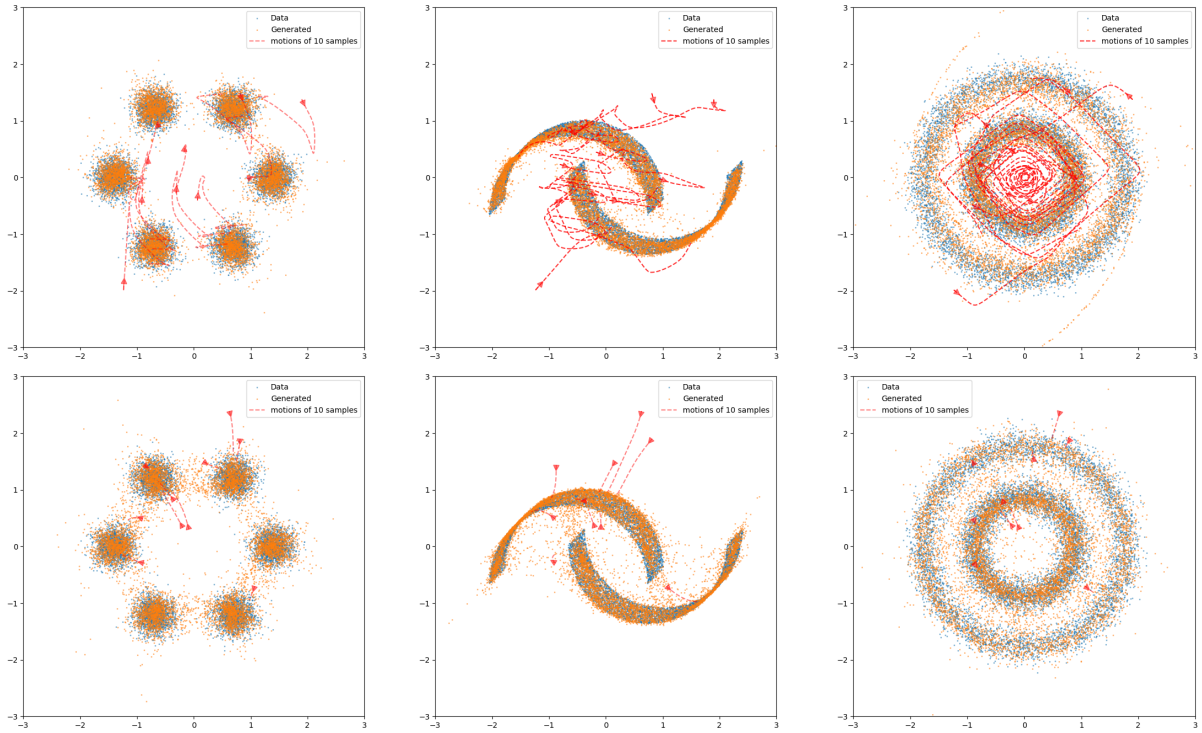


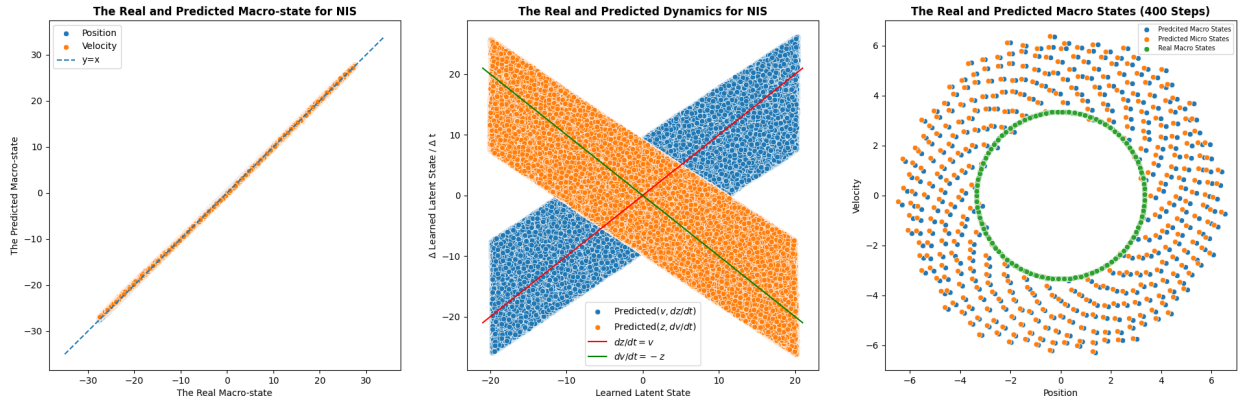
图 9: 上面一行为 Planar Flow 的生成结果；下面一行为 OT Flow 的生成结果

我们可以发现，两类流均有较好的生成效果，而相较于 OT Flow，Planar Flow 的演化过程会相对更加曲折，这是由于 OT Flow 在损失函数中加入了对于演化路程的惩罚。

3.3 信息压缩的实验与结果

为了验证仅将宏观动力学学习器替换为 neural ODE 结构与全部组成部分均替换为 neural ODE 结构的两种 NIS 机器学习架构在信息压缩这一问题上的性能，我们设计了如下实验：

设置初始值均匀分布于 $[-20, 20]$ 的 1280000 个弹簧振子系统数据集，并为他们添加了上文提到的观测误差 ξ，得到了四维的微观状态，利用 sklearn 库中的 odeint 方法计算了下一时间步的微观状态作为微观状态的实际值。我们将数据集划分为 10000 个 batch，batch size 为 128。首先针对仅将宏观动力学学习器替换为 neural ODE，编码器/解码器与原工作相同均为 3 个 RealNVP 模块串接的 NIS 架构进行 5 个 Epoch 的训练，得到结果如图10。

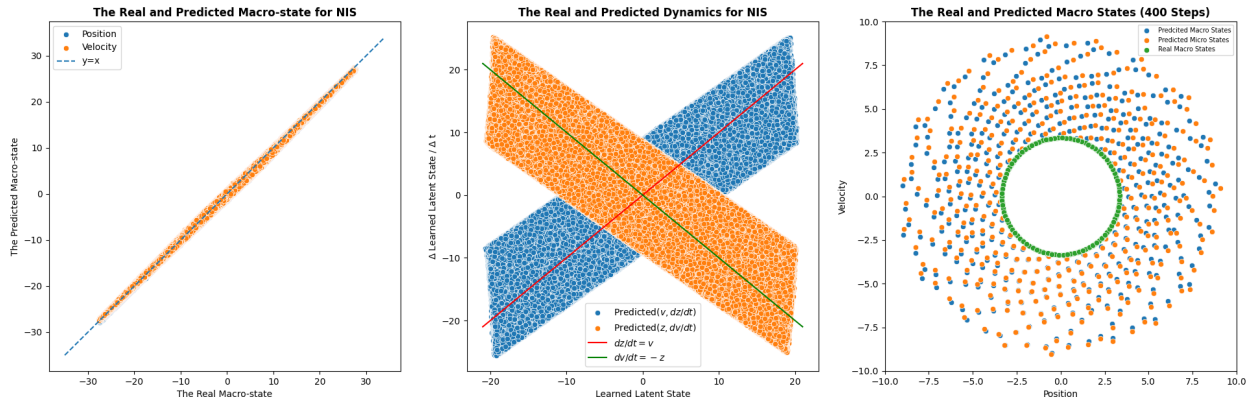


(a) 宏观状态恢复结果　　(b) 宏观动力学学习结果　　(c) 400 个时间步预测结果

图 10: 宏观动力学学习器替换为 neural ODE 的 NIS 信息压缩实验结果

图10a中展示的是信息压缩对宏观状态的恢复，可以看出在宏观状态中无论是位置 (Position) 还是速度 (Velocity) 其预测值与实际值均几乎相等（分布在 $y = x$ 附近），这一结果表明 NIS 架构成功地通过端到端的学习将未知的宏观状态于已知的微观状态之中恢复出来；图10b中展示的是 neural ODE 结构的宏观动力学学习器对宏观状态的动力学的学习结果，其中红色线条与绿色线条展现了弹簧振子模型真实的动力学形式，我们的预测结果在走势上与真实的动力学非常接近，其排布具有一定的宽度是由于我们展示的是 $\frac{MacroState_{t+dt} - MacroState_t}{dt}$，其中 $dt = 1$，由于在一个时间步的尺度上用差分代替微分，因此与实际的微分方程有着一定的误差。但从趋势上看，NIS 对宏观状态的动力学学习效果良好；图10c中展示的是在 400 个时间步上 NIS 对弹簧振子系统演化的预测，其中绿色原点表示弹簧振子各个时间步真实的宏观状态，由于弹簧振子系统的属于简谐运动的过程，因此其相图为一个圆，绿色圆点也正排列在一个圆上，蓝色圆点表示 NIS 预测的各个时间步的宏观状态，黄色圆点表示 NIS 预测的各个时间步的微观状态经过人为解码得到的宏观状态。实验结果表明，在时间步较小的情况下，NIS 能够对弹簧振子的宏观状态做出较为准确的预测，然而当时间步不断增大，由于 NIS 学习到的宏观状态动力学有着一定误差，其对弹簧振子的宏观状态的预测也会产生误差，且误差会随时间步的增加而增加。这一结果与原工作相同 [4]，其产生原因或许是因为在训练时只对一个时间步之后的宏观状态进行了求解，但这并不影响信息压缩任务的主要目的：将未知的宏观状态从已知的微观状态中恢复出来。

针对全部组成部分均替换为 neural ODE 的 NIS 机器学习架构，由于其训练速度较慢，我们在同样的数据集上进行了 2 个 Epoch 的训练，实验结果如图11



(a) 宏观状态恢复结果　　(b) 宏观动力学恢复结果　　(c) 400 个时间步预测结果

图 11: 全部组成部分均替换为 neural ODE 的 NIS 信息压缩实验结果

可见，其各结果均与原工作及仅宏观动力学学习器替换为 neural ODE 结构的 NIS 的实验结果类似，证明了全部组成部分（即编码器/解码器，宏观动力学学习器）均为 neural ODE 的 NIS 架构在信息压缩任务上同样有着良好的性能。

3.4 Neural ODE 应用于分类问题的实验与结果

为验证我们搭建的以 neural ODE 作为特征提取手段的分类模型在分类模型上的具体性能，我们在 MIT-BIH 心律失常数据库上开展了实验。

MIT-BIH 心律失常数据库是由麻省理工学院计算机科学与人工智能实验室（MIT CSAIL）和贝斯以色列女执事医疗中心（Beth Israel Deaconess Medical Center）合作开发的。该数据库被广泛用于研究和开发心电图（ECG）信号的自动分析算法。其数据为具有 187 个采样点的 ECG 数据，共分为 5 类：

- **0 class**: Normal beat
- **1 class**: Supraventricular premature beat
- **2 class**: Premature ventricular contraction
- **3 class**: Fusion of ventricular and normal beat
- **4 class**: Unclassified beat

由于 0 class 的数据过多，导致数据集严重不平衡，因此随机舍弃了 90% 的 0 class 数据，最后形成具有 22330 条数据的训练集与有 5585 条数据的测试集。

设定 Batch Size 为 128，在训练集上对分类模型进行了 10 个 Epoch 的训练，得到在测试集上的分类准确率如图12。

可见，应用 neural ODE 提取特征的分类模型在测试集上表现良好，五类平均准确率达到了 94.02% 的准确率。由于即使对 class 0 数据进行了舍弃数据集仍然展现出了各类别之间数量的不平衡性，因此这一结果也证明此分类模型在数据不平衡的数据上有着较好的鲁棒性。

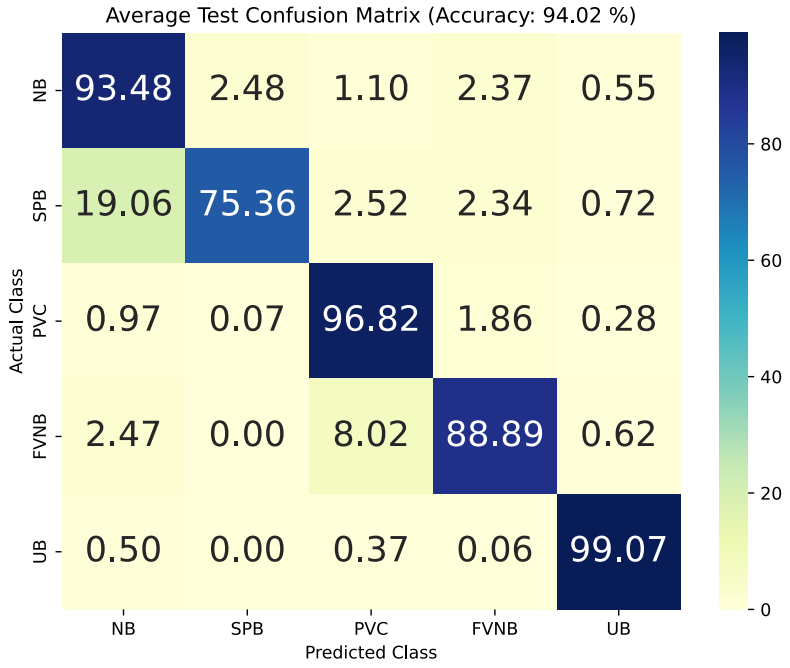


图 12: 测试集上的分类准确率

参考文献

[1] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in neural information processing systems*, 31, 2018.

[2] Danilo Rezende and Shakir Mohamed. Variational inference with normalizing flows. In *International conference on machine learning*, pages 1530–1538. PMLR, 2015.

[3] Derek Onken, Samy Wu Fung, Xingjian Li, and Lars Ruthotto. Ot-flow: Fast and accurate continuous normalizing flows via optimal transport. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 9223–9232, 2021.

[4] Jiang Zhang and Kaiwei Liu. Neural information squeezer for causal emergence. volume 25, page 26. MDPI, 2022.

[5] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density estimation using real nvp. *arXiv preprint arXiv:1605.08803*, 2016.

A Algorithm

A.1 Planar Flow

Algorithm 1 Planar Flow

▷ A neural network maps a single time point t to three parameters in Planar Flow, enhancing the expressive capability of the flow.
function HYPERNETWORK(t)
    return $[\mathrm{u}, \mathrm{w}, b]$
▷ Define the ODE of random variable and the log probability density of it
function ODEFUNC($[\mathrm{z}(t), \log p(\mathrm{z}(t))]$, t):
    $\mathrm{u}, \mathrm{w}, b \leftarrow \mathrm{HyperNetwork}(t)$
    $\frac{d\mathrm{z}(t)}{dt} = \mathrm{u} \cdot \tanh(\mathrm{w}^T \mathrm{z}(t) + b)$
    $\frac{d \log p(\mathrm{z}(t))}{dt} = -\mathrm{tr}\left\{\frac{\partial (d\mathrm{z}(t)/dt)}{\partial \mathrm{z}(t)}\right\}$
    return $\frac{d\mathrm{z}(t)}{dt}, \frac{d \log p(\mathrm{z}(t))}{dt}$
▷ By solving the ODEFunc, we obtain the result of the flow. In short, we can define a function `CNF` to represent the solution of the ODEFunc.
function CNF($[\mathrm{z}(t_0), \log p(\mathrm{z}(t_0))]$, t_0, t_1):
    $[\mathrm{z}(t_1), \log p(\mathrm{z}(t_1))] = \mathrm{ODESolve}([\mathrm{z}(t_0), \log p(\mathrm{z}(t_0))], \mathrm{ODEFunc}, t_0, t_1)$
    return $[\mathrm{z}(t_1), \log p(\mathrm{z}(t_1))]$
▷ Assuming we can sample from the target distribution $p(\mathrm{x})$, we set the terminal state $\mathrm{z}(t_1)$ as the random variable x and the initial state $\mathrm{z}(t_0)$ as the base random variable u following the standard normal distribution. Then we can train the parameters in `HyperNetwork` using a loss function based on the properties of the standard normal distribution, which is equivalent to the KL divergence.
procedure TRAIN($[\mathrm{z}(t_1), \log p(\mathrm{z}(t_1))]$, t_1, t_0)
    $\mathrm{z}(t_1) \leftarrow$ *random variable follows target distribution*
    $\log p(\mathrm{z}(t_1)) \leftarrow 0$
    $[\mathrm{z}(t_0), \log p(\mathrm{z}(t_0))] = \mathrm{CNF}([\mathrm{z}(t_1), \log p(\mathrm{z}(t_1))], t_1, t_0)$
    $l = \mathbb{E}_{p_0(\mathrm{x})}\{\frac{1}{2}\|\mathrm{z}(t_0)\|^2 + \log p(\mathrm{z}(t_0)) + \frac{d}{2}\log(2\pi)\}$; ▷ Copute the loss
    backward.
▷ To generate the target distribution from the base distribution, we simply reverse the ODESolve and sample from the standard normal distribution.
procedure GENERATION($[\mathrm{z}(t_0), \log p(\mathrm{z}(t_0))]$, t_0, t_1)
    $\mathrm{z}(t_0) \leftarrow$ *random variable follows standard normal distirbution*
    $\log p(\mathrm{z}(t_0)) \leftarrow 0$
    $[\mathrm{z}(t_1), \log p(\mathrm{z}(t_1))] = \mathrm{CNF}([\mathrm{z}(t_0), \log p(\mathrm{z}(t_0))], t_0, t_1)$
    return $[\mathrm{z}(t_1), \log p(\mathrm{z}(t_1))]$

A.2 OT Flow

Algorithm 2 OT Flow

▷ According to 13, we define the `ResNet` N with three parameters: d (the dimension of the random variable, with input in $\mathbb{R}^{d+1}$), m (the dimension of the hidden space), and nTh (the number of ResNet layers).
procedure RESNN($d, m,$ nTh)
    ▷ We define Linear Layers in ResNet first, each Linear Layer consists of weights K_j and biases b_0
    `LiearLayers[0]` : $\mathbb{R}^{d+1} \to \mathbb{R}^m$
    `LiearLayers[i]` : $\mathbb{R}^m \to \mathbb{R}^m$, where $i \in \{1, 2, \cdots, \text{nTh} - 1\}$
    `h = 1/(nTh-1)` ▷ Set the step-size in ResNet according to the number of layers
    ▷ We could use the procedure below to perform the function N in the Phi function in 12
    function FORWARD(s)
        `u[0] =` σ`(LinearLayers[0](s))`
        for `i in 1:nTh` **do**
            `u[i] = u[i-1] + 1/(nTh-1) *` σ`(LinearLayers[i](u[i-1]))`
        return `u[nTh-1]`
    ▷ In practice, according to 10 and 12, the weights K_i and biases b_i in the ResNet linear layers are all we need.
    return `LinearLayers`
▷ Define an object called `Phi`, which correspond to the Phi function in 12
procedure PHI($d, m,$ nTh)
    `N = ResNN(`$d, m,$ `nTh)` ▷ `instantiate an ResNN into N`
    ▷ We define the vector w as a linear map from $\mathbb{R}^m$ to $\mathbb{R}$ without bias. Therefore, we can write $\mathrm{w}^T N(\mathrm{s}; \boldsymbol{\theta}_N)$ as `w(N(s))`. And we define the vector b and the scalar c as a linear map from $\mathbb{R}^{d+1}$ to $\mathbb{R}$ with bias. Therefore, we can write $\mathrm{b}^T\mathrm{s} + c$ as `c(s)`: `c(s).weight` represent the $\mathrm{b}(\mathrm{s})$ and `c(s).bias` represent the c
    `w` $\leftarrow$ `Linear (without bias):` $\mathbb{R}^m \to \mathbb{R}$
    `c` $\leftarrow$ `Linear (with bias):` $\mathbb{R}^{d+1} \to \mathbb{R}$
    A $\leftarrow \mathbb{R}^{r \times (d+1)}$ matrix.
    function FORWARD(s)
        return `w(N.forward(s)) + s.T@(A.T@A)@s + c(s)`
    function GRADANDTR(s)
        ▷ According to eq.14 and eq.15, we calculate the gradient of Phi
        `u[0] =` σ`(N.LinearLayers[0](s))`
        for `i in 1:nTh` **do**
            `u[i] = u[i-1] +` σ`(N.LinearLayers[i](u[i-1]))`
        for `i in (nTh-1):-1:0` **do**
            if `i == nTh-1` **then**
                `term = w.weight`
            else
                `term = z[i+1]`
            `z[i] = term`
                `+ N.h*N.LinearLayers[i].weight.T@tanh(N.LinearLayers[i](u[i-1])*term)`
        `grad` $\leftarrow$ `z[0] = N.LinearLayers[0].weight.T@tanh(N.LinearLayers[0](s)*z1)`

▷ According to the eq.16 and eq.17, we calculate the trace of Hessian matrix.
t0 = sum(σ''(N.LinearLayers[0](s)*z[1])*pow(N.LinearLayers[0].weight[:,0:d],2)
J = tanh(N.LinearLayers[0](s))*N.LinearLayers[0].weight[:,0:d]
for i in 1:nTh **do**
  KJ = N.LinearLayers[i].weight@J
  if i == nTh-1 **then**
    term = w.weight
  else
    term = z[i+1]
  temp = N.LinearLayers[i](u[i-1])
  ti = sum(σ''(temp)*term * pow(KJ,2)
  sum_t = t0 + N.h * ti
  J = J + N.h * tanh(temp)
trH ← sum_t + tr(A.T@A[0:d, 0:d])
return grad, trH
procedure TRAINING
PhiObj ← Phi ▷ Instantiate an Phi Network.
▷ Define the ODEFunc to be solved by ODESolver. The input augment_vec is in $\mathbb{R}^{d+3}$ space, where the first d elements represent the random variable $\mathrm{z}(\mathrm{x}, t) \in \mathbb{R}^d$, and the last three elements correspond to $\ell(\mathrm{x}, t)$, $L(\mathrm{x}, t)$, and $R(\mathrm{x}, t)$, respectively.
function ODEFUNC(augment_vec,t)
  z = augment_vec[0:d]
  grad, tr = PhiObj.GradAndTr([z,t])
  dz/dt = -grad[0:d]
  dℓ/dt = -tr
  dL/dt = 1/2*pow(L2Norm(grad[0:d]),2)
  dR/dt = abs(grad[-1] - dL/dt)
  return [dz/dt, dℓ/dt, dL/dt, dR/dt]
loop
  z[0] ← sample from $\rho_0(\mathrm{x})$
  augment_vec_init ← [z[0],0,0,0]
  augment_vec_terminal = ODESolve(augment_vec_init, ODEFunc, 0, T)
  C = 1/2*pow(L2Norm(augment_vec_terminal[0:d]),2)
    - augment_vec_terminal[-3]
    + d/2*log(2π)
  L = augment_vec_terminal[-2]
  R = augment_vec_terminal[-1]
  loss = alph[0]*C + alph[1]*L + alph[2]*R
  loss.backward
procedure GENERATION
z[T] ← sample from standard normal distribution.
augment_vec = ODESolve([z[T],0,0,0],ODEFunc,T,0)
return z[0] = augment_vec[0:d]